

AI Tools & Prompt Usage Summary

Multiple AI tools were used at different stages of the KOHLER AI Bathroom Designer development process. Each tool supported a specific part of the workflow, ranging from initial ideation and visual exploration to implementation, iteration, and technical troubleshooting.

1. ChatGPT — Initial Ideation and Planning

Purpose:

ChatGPT was used during the initial stage of the project to understand the case study, explore possible solution approaches, define the core functionality, and plan the overall website structure.

Brainstorming and Prompt Development

This chat was primarily used to brainstorm the KOHLER AI Bathroom Designer & Planner concept and develop/refine the prompts used to build the prototype.

Initial Concept

The original idea was a website that takes:

- Bathroom dimensions
- Budget
- Preferred design theme
- KOHLER catalogue data

and generates suitable bathroom products with an optional 2D/3D bathroom layout.

Main Ideas Developed

1. Catalogue-driven products

The system should use actual KOHLER products rather than fictional/generated products. Product data should include IDs, categories, prices, dimensions, materials, finishes, colours, styles, features, and catalogue images.

2. Separate AI, recommendation and rendering

The architecture was refined so that AI/computer vision is used mainly for understanding catalogue/reference images, while a deterministic optimizer selects products and Three.js renders them.

Catalogue / Reference Images



Product Data



Recommendation Engine



Product IDs



Asset Mapping



Three.js Renderer

3. Product and 3D asset connection

Each product ID should connect its catalogue information, image, and corresponding 3D asset. The KOHLER 3D Warehouse collection was identified as a potential source for bathroom assets, with kitchen and unrelated models excluded.

4. Bathroom visualization

The bathroom should have dynamically sized walls and floor, visibly use the selected products and tiles, and support rotation, zoom, and pan. Placeholders can be used when an actual model is unavailable.

5. UI direction

The initial prototype was considered too busy, so the prompt was refined toward a minimalistic black/dark interface, with the 3D bathroom as the main element and recommendations/controls in floating side panels.

6. Reference images

The supplied reference images were explicitly included in the final prompt as the primary visual and UX reference for layout, proportions, 3D placement, panels, spacing, typography, interaction style, and overall mood.

Evolution of the Final Prompt

The prototype prompt gradually evolved from a general “AI bathroom generator” into a more specific:

KOHLER catalogue → structured product data → deterministic recommendation → product IDs → 3D asset mapping → interactive Three.js bathroom → minimal reference-based UI

The final prompt also instructed the coding AI to inspect the existing prototype first and reuse useful parts where possible, rather than automatically rebuilding everything from scratch.

Overall, this chat was used as a brainstorming and prompt-development stage, helping define the project's architecture, product-data requirements, 3D asset strategy, UI direction, and the final implementation prompt.

Final combined prompt

All of those requirements were eventually consolidated into the final prototype prompt for the coding AI.

The main evolution was:

Bathroom generator

- KOHLER catalogue-based system
 - structured product database
 - deterministic optimizer
 - 3D asset mapping
 - KOHLER 3D Warehouse assets
 - actual catalogue tiles
 - interactive Three.js bathroom
 - minimal floating-panel UI
 - reference-image-based visual design.
-

Outcome –

Build a working interactive website called **KOHLER AI Bathroom Designer & Planner**.

The goal is to let a user enter their **bathroom dimensions, budget, and preferred design theme**, then automatically create a suitable bathroom design using the **KOHLER catalogue data I provide**.

How the system should work

I will provide KOHLER catalogues/product data containing products such as **tiles, toilets, smart toilets, faucets, showers, vanities, mirrors, bathtubs and accessories**, along with their images and available specifications.

The system should:

1. **Read and structure the catalogue**

- * Identify each product and its category.
- * Assign every product a unique ID.
- * Extract available information such as price, dimensions, finish, material, colour, style and features.
- * Keep the product's catalogue image linked to its ID.

2. **Understand design themes**

- * Use provided theme/reference images to identify design characteristics such as colours, materials, lighting, tile style, mirror shape, vanity style and accessories.
- * Convert these into structured theme attributes.

3. **Optimize the bathroom**

- * Take the user's dimensions, budget and selected theme.
- * Select the best combination of catalogue products.
- * Consider budget, physical space, compatibility and theme/design match.
- * Use a deterministic scoring/optimization system rather than an LLM.

- * Do not invent products or specifications.

4. ****Render the result in 3D****

- * Use ****Three.js**** for an interactive 3D bathroom.
- * Create a floor and three walls.
- * Dynamically resize the room according to the user's dimensions.
- * Automatically place the selected toilet, vanity, shower, mirror, tiles, etc.
- * The user can rotate, zoom and pan around the bathroom.

Each product must have:

```text

Product ID → Product information → Product image → 3D asset

``

If an actual 3D model is unavailable, use a simple appropriate 3D placeholder. The architecture should allow the placeholder to later be replaced by the actual product model.

#### ### User interface

Start with:

**\*\*Design your bathroom\*\***

Then ask:

**\*\*Your Space\*\***

- \* Width

Length

**\*\*Your Budget\*\***

- \* Budget amount

**\*\*Your Style\*\***

- \* Minimalist Modern

Japanese Zen

Classic Luxury

Contemporary

- \* Other provided themes

Then:

**\*\*Generate Bathroom\*\***

After generation, show the interactive 3D bathroom alongside:

- \* selected products

- \* product images

- \* individual prices
- \* total price
- \* remaining budget
- \* theme/design match
- \* basic space-fit information

Allow the user to replace individual products or change the theme, after which the optimizer should update the design and 3D scene.

### ### Important architecture

Keep these separate:

```
```text
```

KOHLER Catalogue

↓

Product/Theme Data

↓

Optimization Engine

↓

Selected Product IDs

↓

3D Asset Mapping

↓

Three.js Bathroom

```
``
```

The AI/computer-vision part should be used for **understanding catalogue/reference images**, not for directly controlling the 3D scene.

The optimizer decides **what products to use**.

Three.js decides **how to display them**.

Technology

Use:

- * React + Vite
- * Three.js
- * JavaScript/TypeScript
- * Local JSON/CSV catalogue data

Keep the prototype frontend-based and suitable for **GitHub Pages** deployment.

Do NOT build

- * No LLM-based product selection

- * No AI-generated 3D bathrooms
- * No perfect photo-to-3D reconstruction
- * No construction/CAD-level accuracy
- * No complicated free-form CAD editor
- * No invented KOHLER products or specifications

Focus on making the **catalogue → optimization → product selection → interactive 3D bathroom** pipeline actually work.

I want to add a couple of things to this prompt - 1. <https://embed-3dwarehouse-classic.sketchup.com/collection/886afdb19cf5a0e12e8ae34ac2aaf4f2/Kohler-Kitchen-Bath-Products> __ i want the recommendation system to use the assets from here only for the design of the bathroom - do not use the ones that dont fit in a bathroom - 2. i want the tiles to be visible and for the styles to be something kohler actually offers - like stone and bathroom specific tiles 3. i want the website to be minimalistic and not cluttered the main page to only be a render in a black background with side panels for the rest of the recommendations would it help if i gave a design image?

2. ChatGPT Image Generation — Visual References

Purpose:

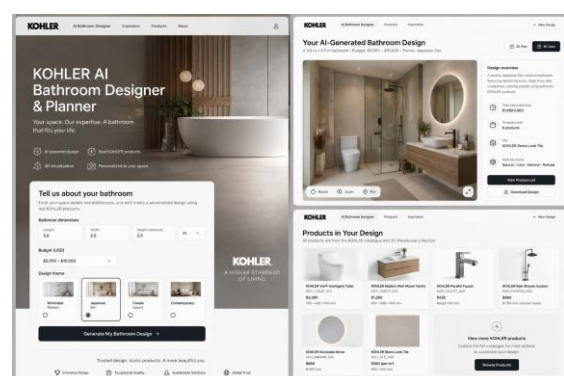
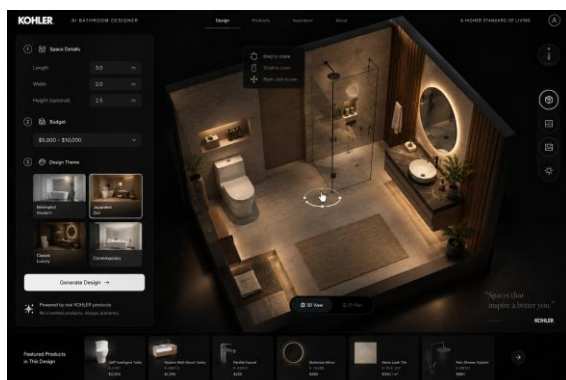
Image generation was used to explore visual concepts and establish references for the user interface and bathroom-design experience before implementation.

Prompt Areas:

- Generating reference concepts for a modern bathroom-planning interface.
- Exploring layouts for the AI designer and product recommendation pages.
- Visualizing a dark-themed interactive 3D bathroom configurator.
- Exploring the presentation of bathroom products alongside an interactive room render.

Outcome:

The generated images were used as visual references rather than final application assets. They helped guide decisions regarding page composition, visual hierarchy, product presentation, and the appearance of the interactive designer.



3. Antigravity — Initial Website Implementation

Purpose:

Antigravity was used to translate the initial requirements and visual direction into the first functional version of the website.

Prompt Areas:

- Creating the initial website structure based on the planned user journey.
- Building the primary UI pages and navigation.
- Establishing the visual design and component structure.
- Creating the initial bathroom designer/rendering interface.
- Integrating the core project concept into a working web prototype.

Outcome:

This produced the initial implementation that served as the foundation for subsequent development and refinement.

4. GitHub Copilot — Implementation Tweaks and Iterative Changes

Purpose:

GitHub Copilot was used during development to assist with incremental code modifications, implementation details, debugging, and refinement of existing functionality.

Prompt Areas:

- Modifying existing React and TypeScript component

General Prompt Approach

This chat was used as an iterative coding and design workflow for the KOHLER bathroom planning prototype.

The prompts generally focused on:

- Diagnosing why the Three.js renderer was blank.
- Checking browser console errors, canvas size, GLB loading, and WebGL behavior.
- Fixing camera framing, rotation, zoom, pan, and 2D view behavior.
- Replacing unreliable GLB assets with procedural Three.js models.
- Building a complete bathroom scene with:
 - Walls and door

- Shower and glass enclosure
- Bathtub option
- Toilet, vanity, mirror, faucet, and plant
- Lighting and tiles
- Making fixture placement respond to room dimensions.
- Adding themes, tile patterns, recommendations, product replacement, and PDF export.
- Cleaning unused files and documenting the project in the README.
- Creating prompts, video, and ppt folders for project deliverables.

How the chat was used:

The chat followed a repeated cycle:

1. Identify an issue from the screenshot or user description.
2. Inspect the relevant source code and assets.
3. Form a likely technical explanation.
4. Apply a focused code change.
5. Run npm run build or browser validation.
6. Refine the result based on the next screenshot or behavior.

Examples included:

- Using browser inspection to discover duplicate Three.js canvases.
- Checking GLB files for mesh and vertex data.
- Identifying that the room canvas had zero dimensions.
- Finding that external product images returned 403 errors.
- Replacing GLB rendering with procedural models when the assets were unreliable.
- Adding a catalogue recommendations page and product-only PDF export.
- Updating and pushing the project README and supporting folders.

Overall, the chat functioned as a collaborative debugging, implementation, visual-design, and documentation process.